

OSSP Skill Week 3

Task 1: Main Loop and Prompt

To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop.

Aim

To build the interactive loop of a shell that displays a prompt, reads input, ignores empty lines and exits when the user types exit.

Program

```
1 #include <stdio.h>
2 #include <string.h>
3 char input[100]
4 int main()
5 {
6     char input[100];
7
8     while (1) {
9         printf("myshell> ");
10        fflush(stdout);
11
12        /* End of input (Ctrl+D) also ends the shell */
13        if (fgetc(input, sizeof(input), stdin) == NULL) {
14            printf("\nExiting shell...\n");
15            break;
16        }
17
18        input[strcspn(input, "\n")] = '\0';
19
20        if (strcmp(input, "exit") == 0) {
21            printf("Exiting shell...\n");
22            break;
23        }
24
25        if (strlen(input) == 0)
26            continue;
27
28        printf("You entered: %s\n", input);
29    }
30
31    return 0;
32 }
```

Output

```
zsh: permission denied: ./
hrushika@Hrushikas-MacBook-Air pjsdj % ./main_loop
myshell> pwd
You entered: pwd
myshell> hello
You entered: hello
myshell> hi
You entered: hi
myshell> WHAT
You entered: WHAT
myshell> 
```

Observation

The prompt was displayed repeatedly, each input was echoed back, empty input was skipped and the loop ended when exit was entered.

Task 2: Copying a File Using System Calls

Aim

To copy the contents of File1.txt to File2.txt using the open(), read(), write() and close() system calls.

1.

Program

File: task2.c

```
1  #include <stdio.h>
2  #include <fcntl.h>
3  #include <unistd.h>
4
5  int main(void)
6  {
7      char buffer[100];
8      ssize_t bytes;
9
10     int source = open("File1.txt", O_RDONLY);
11     if (source < 0) {
12         perror("File1.txt");
13         return 1;
14     }
15
16     int destination = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
17     if (destination < 0) {
18         perror("File2.txt");
19         close(source);
20         return 1;
21     }
22
23     /* Copy the file block by block until end of file */
24     while ((bytes = read(source, buffer, sizeof(buffer))) > 0) {
25         if (write(destination, buffer, bytes) != bytes) {
26             perror("write");
27             close(source);
28             close(destination);
29             return 1;
30         }
31     }
32
33     if (bytes < 0)
34         perror("read");
35
36     close(source);
37     close(destination);
38
39     printf("File copied successfully.\n");
40     return 0;
}
```

Output

```
Hello, this is File1. Operating Systems lab - file copy u
Hello, this is File1. Operating Systems lab - file copy u
● hrushika@Hrushikas-MacBook-Air pjsdj % ./task2
File copied successfully.
✎ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The data was transferred block by block between the two file descriptors. File2.txt was created automatically and contained the same text as File1.txt.

Task 3:

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

Aim

To capture keyboard input character by character using `read()`, `write()` and `termios`, handling Backspace and Enter inside the program.

Program

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define MAX 100

int main(void)
{
    char buffer[MAX];
    char ch;
    int index;
    struct termios oldt, newt;

    /* Save the current terminal settings */
    if (tcgetattr(STDIN_FILENO, &oldt) < 0) {
        perror("tcgetattr");
        return 1;
    }

    /* Disable canonical mode and echo */
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    if (tcsetattr(STDIN_FILENO, TCSANOW, &newt) < 0) {
        perror("tcsetattr");
        return 1;
    }

    while (1) {
        write(STDOUT_FILENO, "\nmysh> ", 7);
        index = 0;

        while (1) {
            if (read(STDIN_FILENO, &ch, 1) != 1) {
                strcpy(buffer, "exit"); /* End of input */
                break;
            }

            if (ch == '\n' || ch == '\r') {
                buffer[index] = '\0';
                write(STDOUT_FILENO, "\n", 1);
                break;
            } else if ((ch == 127 || ch == 8) && index > 0) {
                index--;
                write(STDOUT_FILENO, "\b\b", 3); /* Erase last character */
            } else if (index < MAX - 1 && ch >= 32 && ch < 127) {
                buffer[index++] = ch;
                write(STDOUT_FILENO, &ch, 1); /* Echo character */
            }
        }

        if (strcmp(buffer, "exit") == 0)
            break;

        write(STDOUT_FILENO, "Command: ", 9);
        write(STDOUT_FILENO, buffer, strlen(buffer));
        write(STDOUT_FILENO, "\n", 1);
    }

    /* Restore the original terminal settings */
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
    return 0;
}
```

Output

```
File copied successfully.  
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc task3.c -o task3  
❖ hrushika@Hrushikas-MacBook-Air pjsdj % ./task3  
  
mysh> hello  
Command: hello  
  
mysh> hi  
Command: hi  
  
mysh> pwd  
Command: pwd  
  
mysh> █
```

Observation

Keyboard input capture, Enter key processing and input buffer management were implemented successfully.

Task 4: Escape Sequences and Command History

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

Aim

To use escape sequences for formatted output and to understand how a shell stores commands in history and recalls them with the arrow keys.

Program

```
#include <stdio.h>

int main(void)
{
    printf("\tHello\n");
    printf("Operating\tSystems\n");
    printf("Linux\nProgramming\n");

    return 0;
}
```

Output

```
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc escape.c -o escape
● hrushika@Hrushikas-MacBook-Air pjsdj % ./escape
    Hello
Operating      Systems
Linux
Programming
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

Escape sequences were applied successfully and the working of command history storage and recall was understood.

Task 5: Dynamic Memory Management

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

Aim

To allocate memory at run time, use it safely, release it correctly and verify the absence of memory leaks using Valgrind.

Program

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/mman.h>

struct Node {
    int data;
    struct Node *next;
};

static void write_str(const char *s)
{
    write(STDOUT_FILENO, s, strlen(s));
}

/*
 * Resize a mapped region. Uses mremap() on Linux; on other systems
 * (such as macOS) it maps a new region, copies the data, and unmaps
 * the old one. Returns MAP_FAILED on error, leaving the old mapping intact.
 */
static void *resize_mapping(void *old_addr, size_t old_size, size_t new_size)
{
#ifdef __linux__
    return mremap(old_addr, old_size, new_size, MREMAP_MAYMOVE);
#else
    void *new_addr = mmap(NULL, new_size, PROT_READ | PROT_WRITE,
                          MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (new_addr == MAP_FAILED)
        return MAP_FAILED;

    memcpy(new_addr, old_addr, old_size < new_size ? old_size : new_size);
    munmap(old_addr, old_size);
    return new_addr;
#endif
}

int main(void)
{
    size_t size = 100;

    /* Allocate 100 bytes using mmap() */
    char *buffer = mmap(NULL, size, PROT_READ | PROT_WRITE,
                        MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (buffer == MAP_FAILED) {
        perror("mmap");
        return 1;
    }

    /* Read input using read(), leaving room for the terminator */
    write_str("Enter text: ");
    ssize_t n = read(STDIN_FILENO, buffer, size - 1);
    if (n < 0) {
        perror("read");
        munmap(buffer, size);
        return 1;
    }
    if (n > 0 && buffer[n - 1] == '\n')
        n--;
    buffer[n] = '\0';

    write_str("You entered: ");
    write_str(buffer);
    write_str("\n");
}
```

```
write_str("\n");

/* Resize the memory to 200 bytes */
size_t new_size = 200;
char *resized = resize_mapping(buffer, size, new_size);
if (resized == MAP_FAILED) {
    perror("resize");
    munmap(buffer, size);
    return 1;
}
buffer = resized;
size = new_size;

/* Append safely without overflowing the buffer */
strncat(buffer, " - Memory Resized", size - strlen(buffer) - 1);
write_str("Updated string: ");
write_str(buffer);
write_str("\n");

/* Create a linked list of 5 nodes */
struct Node *head = NULL, *tail = NULL;
for (int i = 1; i <= 5; i++) {
    struct Node *node = malloc(sizeof(*node));
    if (node == NULL) {
        perror("malloc");
        break;
    }
    node->data = i * 10;
    node->next = NULL;
    if (head == NULL)
        head = node;
    else
        tail->next = node;
    tail = node;
}

/* Display the linked list */
printf("Linked List: ");
for (struct Node *cur = head; cur != NULL; cur = cur->next)
    printf("%d -> ", cur->data);
printf("NULL\n");

/* Free the linked list memory */
while (head != NULL) {
    struct Node *next = head->next;
    free(head);
    head = next;
}

/* Release the mapped memory */
munmap(buffer, size);
printf("Memory released successfully.\n");

return 0;
}
```


Output

```
● hrushika@Hrushikas-MacBook-Air pjsdj % ./task5
Enter text: hello
You entered: hello
Updated string: hello - Memory Resized
Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
Memory released successfully.
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

Dynamic memory allocation and correct release of memory were implemented and verified with Valgrind.